

GROOT2 BEHAVIOR TREE EDITOR

Kapsamlı Türkçe Dokümantasyon | TEKNOFEST 2026 Robotaksi

Konu	Groot2 ile Behavior Tree tasarımı, düzenleme ve hata ayıklama
Proje	TEKNOFEST 2026 Robotaksi-Binek Otonom Araç Yarışması
Kapsam	Kurulum · Arayüz · Node Tipleri · Blackboard · BT XML · Debug
Versiyon	Groot2 v2.x BehaviorTree.CPP v4.x ROS 2 Humble/Iron

i BİLGİ

Bu doküman, Groot2 editörünü sıfırdan öğrenmek isteyen ROS 2 geliştiricileri için hazırlanmıştır. Behavior Tree kavramlarından başlayarak kurulum, arayüz kullanımı, tüm node tipleri, blackboard yönetimi, XML formatı ve TEKNOFEST Robotaksi projesine özel ipuçlarını kapsar.

İÇİNDEKİLER

Bölüm	Başlık	Konu
1	Behavior Tree Nedir?	Temel kavramlar, tarihçe, neden BT?
2	Groot2 Kurulumu	Derleme, binary, ROS 2 entegrasyonu
3	Groot2 Arayüzü	Panel ve pencerelerin detaylı açıklaması
4	Node Tipleri	Action, Condition, Control, Decorator — tam liste
5	Blackboard Sistemi	Veri paylaşımı, port'lar, tip güvenliği
6	BT XML Formatı	Şema, TreeNodesModel, port tanımları
7	Nav2 Entegrasyonu	Nav2 node'ları, RecoveryNode, NavigateToPose
8	Hata Ayıklama	ZMQ monitor, log, breakpoint, sık hatalar
9	TEKNOFEST Uygulaması	Senaryo 2 BT mimarisi ve node detayları
10	Büyük Pratik Tablo	Tüm node'lar, portlar, kullanım örnekleri

1 BEHAVIOR TREE NEDİR?

1.1 Tanım ve Temel Fikir

Behavior Tree (BT), ağaç yapısı kullanan bir görev yürütme çerçevesidir. Oyun endüstrisinde NPC yapay zekası için geliştirilmiş; günümüzde robotik, otonom araç ve drone kontrolünde standart haline gelmiştir. BT'nin temel gücü modülerlik ve reaktifliktir: ağaç her adımda (tick) baştan değerlendirilir, koşullar değiştiğinde davranış anında değişir.

1.2 BT vs Sonlu Durum Makinesi (FSM)

Özellik	Behavior Tree	Sonlu Durum Makinesi (FSM)
Modülerlik	Her node bağımsız, yeniden kullanılabilir	Durum geçişleri birbirine bağlı
Reaktiflik	Her tick'te yeniden değerlendirme	Olay tetiklemedikçe statik kalır
Hata kurtarma	RecoveryNode ile yerleşik	Manuel geçişler gerekir
Görselleştirme	Groot2 ile canlı izleme	Graphviz gibi araçlar gerekir
Ölçeklenebilirlik	Alt ağaçlarla sınırsız büyür	Durum sayısı artınca karmaşıklaşır
Paralel davranış	Parallel node ile doğal	Eş zamanlı durum yönetimi zor
Kod tekrarı	Ortak node'lar paylaşılır	Her FSM için yeniden yazılır

1.3 Temel Kavramlar

Kavram	Açıklama
Tick	BT'nin bir değerlendirme döngüsü. Her tick'te kök node'dan başlanır, yapraklar değerlendirir.
SUCCESS	Node görevini başarıyla tamamladı. Üst node'a başarı sinyali gider.
FAILURE	Node görevi başaramadı. Üst node'a başarısızlık sinyali gider.
RUNNING	Node hâlâ çalışıyor, bir sonraki tick'te devam edecek. (uzun süreli görevler için)
Root Node	Ağacın tepesindeki tek giriş noktası. Hep en baştan değerlendirilir.
Leaf Node	Çocuğu olmayan node. Action ve Condition node'ları yapraktır.
Blackboard	Node'lar arası paylaşılan anahtar-değer veri deposu. {goals}, {detected_light} gibi.
Port	Node'un blackboard ile veri alışverişi yaptığı giriş/çıkış noktası.

1.4 Tick Akışı — Nasıl Çalışır?

BT motoru (BehaviorTree.CPP), bir döngüde sürekli olarak **tick()** fonksiyonunu çağırır. Tick, kök node'dan başlayarak ağacı derinlemesine (depth-first) dolaşır. Her node, durumuna göre SUCCESS / FAILURE / RUNNING döndürür ve üst node bu sonuca göre davranışını belirler.

★ İPUCU

Otonom araç uygulamalarında tick sıklığı genellikle 10-50 Hz arasındadır. TEKNOFEST Robotaksi projesinde 10 Hz (her 100ms) yeterlidir: sensör verileri bu hızda gelir ve araç dinamiği daha yavaş tepki verir.

2 GROOT2 KURULUMU

2.1 Sistem Gereksinimleri

Bileşen	Minimum	Önerilen
İşletim Sistemi	Ubuntu 20.04	Ubuntu 22.04 LTS
ROS 2	Foxy	Humble veya Iron
BehaviorTree.CPP	v3.8	v4.x (zorunlu Groot2 için)
Qt	Qt5	Qt6 (Groot2 GUI için)
CMake	3.16	3.22+
GCC	9.x	11.x+
RAM	4 GB	8 GB+

2.2 Kurulum Yöntemi — Binary (Önerilen)

Groot2 için en hızlı yol AppImage binary'sini indirmektir:

BASH

```
1 # Groot2 AppImage indir
2 wget https://github.com/BehaviorTree/Groot2/releases/latest/download/Groot2.AppImage
3
4 # Çalıştırma izni ver
5 chmod +x Groot2.AppImage
6
7 # Başlat
8 ./Groot2.AppImage
9
10 # (İsteğe bağlı) PATH'e ekle
11 sudo mv Groot2.AppImage /usr/local/bin/groot2
```

2.3 BehaviorTree.CPP v4 Kurulumu

BASH

```
1 # Bağımlılıklar
2 sudo apt install libzmq3-dev libboost-dev
3
4 # Kaynak koddan derle
5 git clone https://github.com/BehaviorTree/BehaviorTree.CPP.git -b v4
6 cd BehaviorTree.CPP
7 mkdir build && cd build
8 cmake .. -DCMAKE_BUILD_TYPE=Release
9 make -j$(nproc)
10 sudo make install
```

2.4 ROS 2 Colcon ile Entegrasyon

CMAKE

```
1 # package.xml — bağımlılık ekle
2 <depend>behaviortree_cpp</depend>
3 <depend>nav2_bt_navigator</depend>
4
5 # CMakeLists.txt
6 find_package(behaviortree_cpp REQUIRED)
7 target_link_libraries(my_node behaviortree_cpp::behaviortree_cpp)
8
9 # Workspace'i derle
10 cd ~/ros2_ws
11 colcon build --symlink-install --packages-select my_robot_pkg
12 source install/setup.bash
```

⚠ DİKKAT

Nav2 Humble sürümüyle BehaviorTree.CPP v4'ü birlikte kullanırken dikkatli olun. Nav2'nin kendi BT node'ları v4 API'sine göre derlenmeli ve aynı libray versiyonu kullanılmalıdır. Farklı versiyonlar çalışma zamanında 'register_node' hataları verir.

3 GROOT2 ARAYÜZİ — PANEL VE PENCERELER

Groot2, BT tasarımını görsel olarak yapmanıza ve canlı izlemenize olanak tanır. Arayüz beş ana bölümden oluşur:

Panel	Konum	İşlev	Kısayol
Node Palette	Sol	Kullanılabilir tüm node'ları listeler. Action/Condition/Control/Decorator grupları	
BT Canvas	Merkez	Ana tasarım alanı. Node'ları sürükleyip bırak ile yerleştir, bağla. Orta tuş: kaydır	
Blackboard Monitor	Sağ üst	Tüm blackboard key'lerini ve anlık değerlerini gösterir.	Ctrl+B
Node Properties	Sağ alt	Seçili node'un tüm portlarını ve değerlerini düzenle.	Çift tıkla
Log Console	Alt	BT çalışma logları, hata mesajları, uyarılar.	Ctrl+L

3.1 Node Palette — Sol Panel

Sol panelde ağaç içinde kullanabileceğiniz tüm node'lar listelenir. Node'lar üç kaynaktan gelir:

- **Built-in:** Groot2 ile gelen standart kontrol node'ları (Sequence, Fallback, Parallel, vb.)
- **Nav2:** nav2_bt_navigator paketindeki hazır action node'ları (NavigateToPose, BackUp, Spin...)
- **Custom:** Sizin C++ ile yazdığınız ve XML'de TreeNodesModel içinde tanımladığınız node'lar

3.2 Canvas — Merkez Tasarım Alanı

Eylem	Kısayol / Yöntem
Node ekle	Palette'den canvas'a sürükleyip bırak veya sağ tıkla → Add Node
Node bağla	Sol node'un çıkış noktasına tıkla, sağ node'un giriş noktasına çek
Node sil	Seç → Delete tuşu
Tüm ağacı sığdır	Ctrl+Shift+H veya araç çubuğu → Fit to View
Zoom in/out	Ctrl+Scroll veya + / - tuşları
Canvas kaydır	Orta fare tuşu basılı tutarak sürükleyip bırak
Alt ağaç katla	Node üzerindeki üçgen simgeye tıkla
Kopyala/Yapıştır	Ctrl+C / Ctrl+V (subtree'ler dahil)
Geri Al / Yinele	Ctrl+Z / Ctrl+Y

3.3 Canlı İzleme Modu (ZMQ Monitor)

Groot2'nin en güçlü özelliği canlı izlemedir. BT çalışırken Groot2'ye bağlanarak hangi node'un aktif olduğunu, hangi node'un SUCCESS/FAILURE döndürdüğünü ve blackboard değerlerini gerçek zamanlı görebilirsiniz.

CPP

```

1 # C++ kodunda ZMQ publisher başlat
2 #include "behaviortree_cpp/loggers/bt_zmq_publisher.h"
3
4 BT::Tree tree = factory.createTreeFromFile(bt_file);
5
6 // ZMQ publisher – varsayılan port: 1666
7 BT::PublisherZMQ zmq_publisher(tree);
8
9 // Nav2 bt_navigator.yaml'da
10 bt_navigator:
11   enable_groot_monitoring: true
12   groot_zmq_publisher_port: 1666
13   groot_zmq_server_port: 1667

```

Groot2'de bağlantı kurmak için: **Monitor** → **Connect to ZMQ** → **ws://localhost:1666**

3.4 Canvas Renk Kodları

Renk	Node Tipi	Açıklama
Yeşil (#2ecc71)	SUCCESS	Node başarıyla tamamlandı
Kırmızı (#e74c3c)	FAILURE	Node başarısız oldu
Sarı/Turuncu (#f39c12)	RUNNING	Node hâlâ çalışıyor
Gri (#95a5a6)	IDLE	Henüz tick gelmedi
Mavi (#3498db)	Action Node	Düzleştirme tasarımıda
Mor (#9b59b6)	Condition Node	Tasarımda
Turuncu (#e67e22)	Control Node	Tasarımda
Kahverengi (#795548)	Decorator Node	Tasarımda

4 NODE TİPLERİ — TAM BAŞVURU KILAVUZU

4.1 Control Node'ları

Control node'ları bir veya daha fazla çocuğa sahip olabilir. Çocuklardan aldıkları SUCCESS/FAILURE sinyallerine göre kendi dönüş değerlerini belirlerler.

Node	Çocuk Sayısı	Dönüş Mantığı	Kullanım
Sequence	1..N	Tüm çocuklar SUCCESS → SUCCESS Biri FAILURE → FAILURE (durur)	Sıralı görevler: Git → Dur →
ReactiveSequence	1..N	Her tick'te baştan değerlendirir. Biri FAILURE → FAILURE	Koşul değiştiğinde anında t
Fallback (Selector)	1..N	Biri SUCCESS → SUCCESS Tümü FAILURE → FAILURE	Alternatif planlar denemek
ReactiveFallback	1..N	Her tick'te baştan değerlendirir. Biri SUCCESS → SUCCESS	Öncelikli reaktif kontrol
Parallel	1..N	M çocuk SUCCESS → SUCCESS N-M+1 çocuk FAILURE → FAILURE	Eş zamanlı görevler
RecoveryNode	2	Çocuk 1 FAILURE → Çocuk 2 (recovery) retry sayısınca tekrar dener	Hata kurtarma döngüsü
IfThenElse	2-3	Çocuk 1 SUCCESS → Çocuk 2 Çocuk 1 FAILURE → Çocuk 3	Koşullu dallanma
WhileDoElse	2-3	Çocuk 1 SUCCESS → Çocuk 2 (döngü) Çocuk 1 FAILURE → Çocuk 3	Koşula bağlı döngü
ManualSelector	1..N	Kullanıcı elle seçim yapar (test/debug için)	Groot2'de manuel kontrol

⚠ DİKKAT

Sequence vs ReactiveSequence: Normal Sequence, bir çocuğu RUNNING durumda bırakır ve bir sonraki tick'te oradan devam eder. ReactiveSequence ise her tick'te BAŞTAN başlar.

NavigateThroughPoses gibi uzun süren action'lar için Sequence kullanın. TrafficLightCondition gibi anlık kontroller için ReactiveFallback içinde kullanın.

4.2 Decorator Node'ları

Decorator node'ları tam olarak 1 çocuğa sahiptir ve çocuğun dönüş değerini dönüştürür.

Decorator	İşlev	Tipik Kullanım
Inverter	SUCCESS↔FAILURE yer değiştirir	Koşulun tersini almak
ForceSuccess	Her zaman SUCCESS döner	Opsiyonel görevler (Tünel: +200 pu
ForceFailure	Her zaman FAILURE döner	Test / debug
Repeat	N kez RUNNING tutar, sonra SUCCESS	Belirli sayıda tekrar
RetryUntilSuccessful	Çocuk FAILURE → tekrar dene (max N)	Yeniden deneme mantığı

Timeout	Süre aşılsa FAILURE	Zaman sınırlı görevler
Delay	Çalışmadan önce X ms bekler	Zamanlı tetikleme
SingleTrigger	Çocuğu sadece 1 kez çalıştırır	InitMissionAction — GEOJSON tek o
KeepRunningUntilFailure	Çocuk FAILURE dönene kadar RUNNING	Sürekli döngü
SubTree	Başka bir XML ağacını içe aktarır	Modüler BT tasarımı

4.3 Yaprak Node'lar — Action ve Condition

Tip	Karakter	Dönüş	Örnek
Action	Aktif eylem gerçekleştir Async çalışabilir	SUCCESS / FAILURE / RUNNING	NavigateToPose, ParkingAc
Condition	Anlık durum sorgular Async OLMAZ — hemen döner	SUCCESS / FAILURE (RUNNING VERMEZ!)	TrafficLightCondition, IsStuc

× HATA

KURAL: Condition node'ları ASLA RUNNING döndürmez. Anlık bir durumu sorgular ve hemen SUCCESS ya da FAILURE verir. Eğer async bir kontrol gerekiyorsa (örn. servis çağırısı), bunu Action node'u olarak implemente edin.

5 BLACKBOARD SİSTEMİ — VERİ PAYLAŞIMI

Blackboard, BT içindeki tüm node'ların ortak kullandığı bir anahtar-değer deposudur. ROS 2'deki parametre sunucusuna benzer ama çok daha hızlıdır ve tip güvenliği sağlar. Node'lar birbirleriyle doğrudan haberleşmek yerine blackboard üzerinden veri paylaşır.

5.1 Port Türleri

Port Türü	C++ Makrosu	XML Etiket	Açıklama
Input Port	BT_INPUT_PORT(T, name, desc)	<input_port name='..'>	Node blackboard'dan
Output Port	BT_OUTPUT_PORT(T, name, desc)	<output_port name='..'>	Node blackboard'a y
Bidirectional	BT_BIDIRECTIONAL_PORT(T, name, desc)	Her ikisi de	Hem okur hem yazar

5.2 C++ Node İmplementasyonu

CPP

```

1 // Örnek: TrafficLightCondition node'u
2 class TrafficLightCondition : public BT::ConditionNode {
3 public:
4     TrafficLightCondition(const std::string& name,
5                           const BT::NodeConfig& config)
6         : BT::ConditionNode(name, config) {}
7
8     // Port tanımları - XML TreeNodesModel ile eşleşmeli
9     static BT::PortsList providedPorts() {
10         return { BT::InputPort<std::string>("detected_light") };
11     }
12
13     BT::NodeStatus tick() override {
14         auto light = getInput<std::string>("detected_light");
15         if (!light) return BT::NodeStatus::FAILURE;
16         return (light.value() == "RED") ? BT::NodeStatus::SUCCESS
17                                         : BT::NodeStatus::FAILURE;
18     }
19 };
20
21 // Ana programda factory'e kaydet
22 factory.registerNodeType<TrafficLightCondition>("TrafficLightCondition");

```

5.3 TEKNOFEST Senaryo 2 — Blackboard Key Sözlüğü

Key	Tip	Yazar	Okuyan	Değerler
{goals}	vector<PoseStamped>	editMissionAction	NavigateThroughPoses	start→park_giris
{detected_obstacle}	custom msg	Lidar/kamera node	DynamicObstacleCondition	true/false veya mesafe

{detected_light}	string	Kamera/algı node	TrafficLightCondition	"RED" "GREEN" ""
{sign_stop}	bool	YOLO algı node	StopSignCondition	true false
{sign_bus_stop}	bool	YOLO algı node	BusStopCondition	true false
{sign_park}	bool	YOLO algı node	ParkSignCondition	true false
{sign_pedestrian}	bool	YOLO algı node	PedestrianCrossingCondition	true false
{sign_roundabout}	bool	YOLO algı node	RoundaboutCondition	true false
{sign_tunnel}	bool	YOLO algı node	TunnelCondition	true false
{sign_pass_by}	string	YOLO algı node	PassByCondition/Action	"right" "left" ""
{sign_no_entry}	bool	YOLO algı node	NoEntryCondition/Action	true false
{sign_forbidden}	string	YOLO algı node	ForbiddenTurnCondition	"right" "left" ""
{sign_mandatory}	string	YOLO algı node	MandatoryTurnCondition	"TT-35X:yön" ""
{current_vertex}	int	Nav2 topoloji	NoEntry/ForbiddenTurn/Mandatory...	0, 1, 2, 3...
{next_vertex}	int	NoEntry/ForbiddenTurn/Mandatory...	NavigateThroughPoses	rota vertex indeksi

6 BT XML FORMATI — ŞEMA VE KURALLAR

6.1 Temel XML Yapısı

XML

```

1 <?xml version="1.0"?>
2 <root main_tree_to_execute="MainTree">
3
4   <!-- Behavior Tree tanımı -->
5   <BehaviorTree ID="MainTree">
6     <Sequence>
7       <Condition ID="IsRobotReady"/>
8       <Action ID="StartNavigation" goal="{target_pose}"/>
9     </Sequence>
10  </BehaviorTree>
11
12  <!-- Node modeli — Groot2 bu bölümü okur -->
13  <TreeNodesModel>
14    <Condition ID="IsRobotReady">
15      <!-- Port yoksa boş bırakılabilir -->
16    </Condition>
17    <Action ID="StartNavigation">
18      <input_port name="goal">Hedef pose. BB key: {target_pose}</input_port>
19    </Action>
20  </TreeNodesModel>
21
22 </root>

```

6.2 TreeNodesModel — Zorunlu mu?

TreeNodesModel bölümü Groot2 için zorunludur ancak BehaviorTree.CPP runtime için **opsiyoneldir**. Bu bölüm olmadan:

- Groot2 node'larınızı palette'de gösteremez
- Port bilgilerini bilemez ve bağlantıları doğrulayamaz
- Çalışma zamanında node'larınızı factory'den bulur (C++ registerNodeType)

★ İPUCU

İyi pratik: TreeNodesModel'i her zaman yazın. Hem Groot2 editöründe çalışmanızı kolaylaştırır hem de projenin dokümantasyonu görevi görür. Port açıklamalarına TR açıklamalar eklemeniz hakem sunumlarında büyük avantaj sağlar.

6.3 Port Tanımı Kuralları

XML

```
1 <TreeNodesModel>
2   <Action ID="MandatoryTurnAction">
3
4     <!-- Input Port: Node bu değeri blackboard'dan OKUR -->
5     <input_port name="sign_data">
6       Tabela kodu ve yön. Format: "TT-35X:yön"
7       Örnek: TT-35a:right | TT-35d:forward_right
8     </input_port>
9
10    <!-- Input Port: Mevcut vertex indeksi -->
11    <input_port name="current_vertex">
12      Topoloji haritasındaki mevcut dugum indeksi
13    </input_port>
14
15    <!-- Output Port: Node bu değeri blackboard'a YAZAR -->
16    <output_port name="next_vertex">
17      Mecburi yön sonrası hedef vertex. NoEntryAction ile paylaşılır.
18    </output_port>
19
20  </Action>
21 </TreeNodesModel>
```

7 NAV2 ENTEGRASYONU — HAZIR NODE'LAR

Nav2 (Navigation2), ROS 2 için kapsamlı bir navigasyon çerçevesidir ve BT tabanlı görev yönetimi sunar. Groot2 ile Nav2 BT node'larını görselleştirebilir, düzenleyebilir ve canlı izleyebilirsiniz.

7.1 Temel Nav2 Action Node'ları

Node	Paket	Açıklama	Önemli Port
NavigateToPose	nav2_bt_navigator	Tek hedefe navigate et	goal: PoseStamped
NavigateThroughPoses	nav2_bt_navigator	Waypoint listesinden geç	goals: vector<PoseStamped>
ComputePathToPose	nav2_planner	Rota hesapla (hareket etme)	goal, start → path
FollowPath	nav2_controller	Hesaplanan rotayı takip et	path → izle
Spin	nav2_behaviors	Yerinde dön	spin_dist: radyan
BackUp	nav2_behaviors	Geri git	backup_dist: m
DriveOnHeading	nav2_behaviors	Belirli açıda ilerle	dist, speed, heading
Wait	nav2_behaviors	Belirli süre bekle	wait_duration: sn
ClearEntireCostmap	nav2_costmap	Costmap'ı tamamen temizle	server_name
ClearCostmapAroundRobot	nav2_costmap	Çevre costmap temizle	dist: m

7.2 Temel Nav2 Condition Node'ları

Node	Açıklama	SUCCESS Koşulu
IsStuck	Araç ilerleme kaydediyor mu?	İlerleme yoksa SUCCESS (stuck)
GoalReached	Hedefe ulaşıldı mı?	Araç hedefin tolerance m içinde
GoalUpdated	Hedef değişti mi?	Yeni goal blackboard'a yazılmışsa
IsBatteryLow	Batarya kritik mi?	Batarya threshold altındaysa
TransformAvailable	TF dönüşümü var mı?	İki frame arası dönüşüm mevcutsa
TimeExpired	Süre doldu mu?	Verilen süre geçmişse
DistanceTraveled	Mesafe katedildi mi?	Belirli mesafe kaydedildiyse
InitialPoseReceived	Başlangıç pozu verildi mi?	AMCL initial pose alındıysa

7.3 bt_navigator.yaml Yapılandırması

YAML

```
1 # nav2_params.yaml
2 bt_navigator:
3   ros__parameters:
4     use_sim_time: false
5     global_frame: map
6     robot_base_frame: base_link
7     odom_topic: /odom
8
9   # Senaryo 2 için BT dosyası
10   default_nav_through_poses_bt_xml: /path/to/round2_bt_scenario2_final.xml
11
12   # Groot2 canlı izleme
13   enable_groot_monitoring: true
14   groot_zmq_publisher_port: 1666
15   groot_zmq_server_port: 1667
16
17   # Plugin listesi (custom node'lerinizi ekleyin)
18   plugin_lib_names:
19     - nav2_compute_path_to_pose_action_bt_node
20     - nav2_navigate_through_poses_action_bt_node
21     - nav2_back_up_action_bt_node
22     - nav2_spin_action_bt_node
23     - nav2_wait_action_bt_node
24     - nav2_is_stuck_condition_bt_node
25     - my_robot_bt_nodes # Custom node'larınız
```


8 HATA AYIKLAMA — SIK KARŞILAŞILAN SORUNLAR

8.1 Groot2 Bağlantı Sorunları

Hata	Olası Neden	Çözüm
'Connection refused' hatası	ZMQ publisher başlatılmamış Port kapalı	enable_groot_monitoring: true yap Port 1666'nın açık olduğunu doğrula
Node'lar palette'de görünmüyor	TreeNodeModel eksik XML yüklenmemiş	XML'i File → Load BT ile aç TreeNodeModel bölümünü ekle
'Unknown node type' hatası	Factory'e kayıt yapılmamış Plugin listesinde yok	factory.registerNodeType<>() bt_navigator.yaml plugin ekle
Ağaç canlı izlenmiyor	BT çalışmıyor Farklı port	ros2 topic echo /bt_navigator Groot2'de portu kontrol et
Port bağlantısı yok	TreeNodeModel'de port eksik Tip uyumsuzluğu	Port tanımını kontrol et C++ ve XML tipleri eşleşmeli
Blackboard değer güncellenmiyor	Topic subscribe yok Yanlış key adı	Algı node'unun publish yaptığını test et {key} adlarını karşılaştır

8.2 BT Davranış Sorunları

Sorun	Nedeni	Çözüm
Araç ışıktaki durmuyor	TrafficLightCondition FAILURE dönüyor detected_light key boş	YOLO node'unun publish ettiğini kontrol et ros2 topic echo /traffic_light_status
Park çok erken başlıyor	ParkSignCondition çok hassas Yanlış konumda tabela	Confidence threshold artır Geometry mesafe filtresi ekle
Recovery çalışmıyor	IsStuck eşiği çok yüksek Recovery node bağlı değil	progress_checker_plugin ayarla BT yapısını kontrol et
NavigateThroughPoses donuyor	Goals listesi boş InitMissionAction başarısız	InitMissionAction loglarına bak GEOJSON dosyasını doğrula
Tünel tekrar tetikleniyor	Tek seferlik flag yok TunnelCondition hâlâ TRUE	C++ içinde static bool tunnel_used ekle {sign_tunnel} key'ini sıfırla
Döner kavşakta araç donuyor	RoundaboutAction RUNNING'de kalıyor Nav2 timeout aşıldı	RoundaboutAction'ı SUCCESS döndür Nav2 haritasını kontrol et

8.3 Debug İpuçları

- **Groot2 Breakpoint:** Canvas'ta node'a sağ tıkla → Set Breakpoint. BT o node'a gelince durur.
- **Kısmi Ağaç Testi:** Sadece bir subtree'yi ForceSuccess/ForceFailure ile izole ederek test edin.
- **Log Seviyesi:** export RCUTILS_LOGGING_SEVERITY_THRESHOLD=DEBUG ile tam log alın.
- **Blackboard Dump:** C++ içinde tree.rootBlackboard()->debugMessage() ile anlık değerleri yazdırın.
- **Simülasyon Önce:** Gazebo veya CARLA simülasyonunda %100 test edin, sonra gerçek araçla geçin.
- **BT Logları:** BT::FileLogger ile BT tick geçmişini dosyaya yazıp Groot2 ile oynayın.

9 TEKNOFEST 2026 — SENARYO 2 BT MİMARİSİ

9.1 Senaryo 2 Tanımı ve Kapsam

Senaryo 2, dinamik trafik ortamında park görevini kapsar. Yolcu indirme/bindirme bu senaryoda **yoktur**. Araç başlangıç noktasından park bölgesine tüm trafik kurallarına uyarak ulaşmalıdır.

Kriter	Durum	Puan
Trafik Işığы Uyumu	AKTİF	+60 (kırmızı) / +40 (yeşil)
Dinamik Engel Sakınma	AKTİF	+50 / DQ (çarpma)
Statik Engel Sakınma	AKTİF	+50 / DQ (çarpma)
Trafik Tabelaları	AKTİF	+50 / -50 (ihlal)
Şerit İhlali	DEĞERLENDİRİLİR	+50 / -50 (ihlal başına)
Park Bölgesine Ulaşma	ZORUNLU	+20
Park Görevi	ZORUNLU	+80 (tam) / +20 (eksik)
Tünel Geçışı	OPSİYONEL	+200 (tek seferlik)
Yolcu İndirme/Bindirme	YOK	— (Senaryo 2'de yok)

9.2 BT Ağaç Yapısı Özeti

XML

```

1 Sequence (dis kabuk)
2   └─ SingleTrigger
3     └─ InitMissionAction(goals={goals})
4         # GEOJSON yukler: start → park_giris
5         # Yolcu noktasi YOK (Senaryo 2)
6     |
7     └─ RecoveryNode(number_of_retries=3)
8         └─ ReactiveFallback # Ana gorev dongusu
9             └─ P1: Sequence # Dinamik Engel (EN YUKSEK ONCELIK)
10                 └─ DynamicObstacleCondition({detected_obstacle})
11                     └─ Wait(0.5)
12                 |
13                 └─ P2: Sequence # Trafik Isigi
14                     └─ TrafficLightCondition({detected_light})
15                         └─ Wait(0.5)
16                     |
17                     └─ P3: Fallback # Trafik Tabelasi (10 tabela)
18                         └─ StopSign(TT-2) → StopSignAction
19                         └─ BusStop(B-22) → BusStopAction
20                         └─ ParkSign(P-3a) → ParkingAction
21                         └─ Pedestrian(B-14a) → SlowDownAction
22                         └─ Roundabout(TT-37) → RoundaboutAction(stub)
23                         └─ Tunnel(B-49a) → TunnelAction(+200p)
24                         └─ PassBy(TT-36a/b) → PassByAction(direction)
25                         └─ NoEntry(TT-4) → NoEntryAction
26                         └─ ForbiddenTurn(TT-26a/b) → ForbiddenTurnAction
27                         └─ MandatoryTurn(TT-35a-h) → MandatoryTurnAction
28                     |
29                     └─ P4: NavigateThroughPoses({goals})
30                         # Buraya ANCAK engel+isik+tabela yoksa gelinir
31                 |
32             └─ Recovery Sequence
33                 └─ IsStuck
34                 └─ BackUp(dist=0.3m, speed=0.05m/s)
35                 └─ Spin(1.57 rad = 90 derece)

```

9.3 Öncelik Sırası Mantığı

ReactiveFallback her tick'te SOLDAN SAĞA tüm çocukları dener. İlk SUCCESS dönen çocuk kazanır ve o tick'te başka çocuk çalışmaz. Bu sayede:

Öncelik	Durum	Davranış
P1 — EN YÜKSEK	Dinamik engel algılandı	Her şey durur, 0.5sn bekle
P2	Kırmızı ışık var	Tüm navigasyon durur, ışık yeşile dönene kadar b
P3	Tabela algılandı	İlgili tabela action'ı çalışır

P4 — EN DÜŞÜK	Hiçbiri yok	Normal navigasyon devam eder
---------------	-------------	------------------------------

✓ **ÖNEMLİ**

Örnek senaryo: Araç kırmızı ışıktaki bekliyor (P2 aktif). Aynı anda önüne bir araç giriyor (P1 devreye girer, P2 devre dışı). Engel geçince P2 yeniden aktif. Işık yeşile dönünce P4 (navigasyon) devam eder. ReactiveFallback bu geçişi otomatik yönetir — ekstra kod gerekmez.

10 TAM NODE REFERANS TABLOSU — SENARYO 2

Aşağıdaki tablo, Senaryo 2 BT'sinde kullanılan tüm node'ları, portlarını, C++ sınıf isimlerini ve implementasyon notlarını içerir.

Node ID	Tip	Blackboard Portu	Şartname	C++ Notu
InitMissionAction	Action	OUT: {goals}	—	GEOJSON parse, PoseStamp
DynamicObstacleCondition	Condition	IN: {detected_obstacle}	Sayfa 24, +50/-DQ	Lidar mesafe eşiği
TrafficLightCondition	Condition	IN: {detected_light} "RED" "GREEN" " "	Sayfa 27-28	+60 kırmızı +40 yeşil
StopSignCondition	Condition	IN: {sign_stop} bool	TT-2, Sayfa 30	YOLO confidence >0.7
StopSignAction	Action	—	-50 ihlal	cmd_vel=0, 3sn bekle
BusStopCondition	Condition	IN: {sign_bus_stop} bool	B-22, Sayfa 23	YOLO confidence >0.7
BusStopAction	Action	—	Durma ihlali	cmd_vel=0, param sn
ParkSignCondition	Condition	IN: {sign_park} bool	P-3a, Sayfa 28	+80 başarılı park
ParkingAction	Action	—	+80/+20/-20	3dk timer, spot seç
PedestrianCrossingCondition	Condition	IN: {sign_pedestrian} bool	B-14a	Görüş alanında flag
SlowDownAction	Action	—	—	max_vel*0.5, restore_speed
RoundaboutCondition	Condition	IN: {sign_roundabout} bool	TT-37, Sayfa 20	YOLO algı
RoundaboutAction	Action	—	—	return SUCCESS; (stub)
TunnelCondition	Condition	IN: {sign_tunnel} bool	B-49a, Sayfa 30	+200 puan opsiyonel
TunnelAction	Action	—	+200 (tek)	static bool flag, hız düşür
PassByCondition	Condition	IN: {sign_pass_by} "right" "left" " "	TT-36a/b	İki tabela ayrı id
PassByAction	Action	IN: direction "right" "left"	—	Costmap taraf engelle
NoEntryCondition	Condition	IN: {sign_no_entry} bool	TT-4	Yol kapalı
NoEntryAction	Action	IN: current_vertex OUT: next_vertex	—	Kenarı lethal işaretle
ForbiddenTurnCondition	Condition	IN: {sign_forbidden} "right" "left" " "	TT-26a/b	Dönüş yasağı
ForbiddenTurnAction	Action	IN: current_vertex OUT: next_vertex	—	Costmap kısıtı
MandatoryTurnCondition	Condition	IN: {sign_mandatory} "TT-35X:yon" " "	TT-35a-h (7 var)	7 format parse et
MandatoryTurnAction	Action	IN: sign_data, current OUT: next_vertex	—	split(':'), Nav2 replanning

NavigateThroughPoses	Action (Nav2)	IN: {goals} behavior_tree	—	Nav2 built-in
IsStuck	Condition (Nav2)	—	—	progress_checker plugin
BackUp	Action (Nav2)	backup_dist, backup_speed	—	Nav2 built-in
Spin	Action (Nav2)	spin_dist (rad)	—	Nav2 built-in
Wait	Action (Nav2)	wait_duration (sn)	—	Nav2 built-in

★ İPUCU

YOLO Algı Node'u (Ayrı geliştirme):

Tüm {sign_*} blackboard key'leri, perception pipeline'inızın YOLOv8/v11 modelinden gelen tespitleri işleyip ROS 2 topic'e publish etmesiyle dolar. Önerilen topic yapısı: /traffic_signs (custom msg) → blackboard_writer_node → Blackboard. Confidence threshold: 0.70 (gerçek araç), 0.60 (simülasyon). Algı node'u bt_navigator lifecycle'ına bağlanmalı ve her 100ms'de publish etmelidir.

⚠ DİKKAT

Minimum Hız Zorunluluğu (Şartname Sayfa 31):

Araç, normal seyrinde ortalama yürüyüş hızının üzerinde seyretmelidir (>1.4 m/s önerilen). Nav2 controller'da min_vel_x: 0.0 yerine 0.1 ayarlamak ve max_vel_x: 2.0 üzerinde tutmak puanlamayı etkiler. SlowDownAction'da minimum 0.5 m/s altına inmeyin.

HIZLI BAŞVURU KARTI

Groot2 kısayolları, sık kullanılan komutlar ve kritik hatırlatmalar

Groot2 Kısayolu	İşlev	ROS 2 Komutu	Açıklama
Ctrl+Z / Ctrl+Y	Geri al / Yinele	ros2 launch nav2_bringup bringup_launch	Nav2 başlat
Ctrl+C / Ctrl+V	Kopyala / Yapıştır (subtree)	ros2 topic echo /bt_status	BT durumu izle
Ctrl+Shift+H	Tüm ağacı ekrana sığdır	ros2 topic echo /diagnostics	Sistem sağlığı
Delete	Seçili node'u sil	ros2 param set /bt_navigator enable_groot2	Groot2 etkinleştir
Çift tıkla	Node Properties aç	ros2 run groot2 Groot2	Groot2 başlat (ROS)
Sağ tıkla	Context menü (Add/Remove)	ros2 bag record -a	Tüm topic'leri kaydet
Ctrl+S	BT XML olarak kaydet	ros2 bag play bag_name.db3	Kaydı oynat
Ctrl+O	BT XML yükle	colcon build --symlink-install	Workspace derle
F5	ZMQ bağlantısını yenile	source install/setup.bash	Ortamı yükle

SENARYO 2 — KRİTİK HATIRLATMALAR

1. PassengerAction BU SENARYODA YOK — {goals} listesinde yolcu noktası (gorev_X) bulunmaz.
2. TunnelAction için static bool tunnel_used C++ flag'i ZORUNLU — şartname tek seferlik puan der.
3. RoundaboutAction stub'ı — sadece SUCCESS döner, Nav2 haritadan geçişi yönetir.
4. BusStopAction ve StopSignAction ayrı node — bekleme süreleri farklı C++ parametresi.
5. MandatoryTurnAction sign_data'yı 'TT-35X:yon' formatında parse eder — 7 varyant.
6. SingleTrigger olmadan InitMissionAction her tick'te GEOJSON okur — çok yavaşlatır.
7. RecoveryNode number_of_retries=3 — kapalı yollar birden fazla recovery gerektirebilir.
8. SlowDownAction tabela gözden kaybolunca otomatik durur — C++ restore_speed() zorunlu.
9. Algı node'u her 100ms publish etmeli — BT tick sıklığıyla eşleşmeli (10 Hz).

Bu dokümantasyon TEKNOFEST 2026 Robotaksi-Binek Otonom Araç Yarışması şartnamesine (v1.0, 02.01.2026) göre hazırlanmıştır. Groot2 v2.x ve BehaviorTree.CPP v4.x ile uyumludur. Nav2 Humble/Iron sürümleri için geçerlidir.